

GTA-2: A VLM Framework for Zero-Shot Robot Manipulation via Object-Centric Task-Axis Controllers

Anonymous Authors

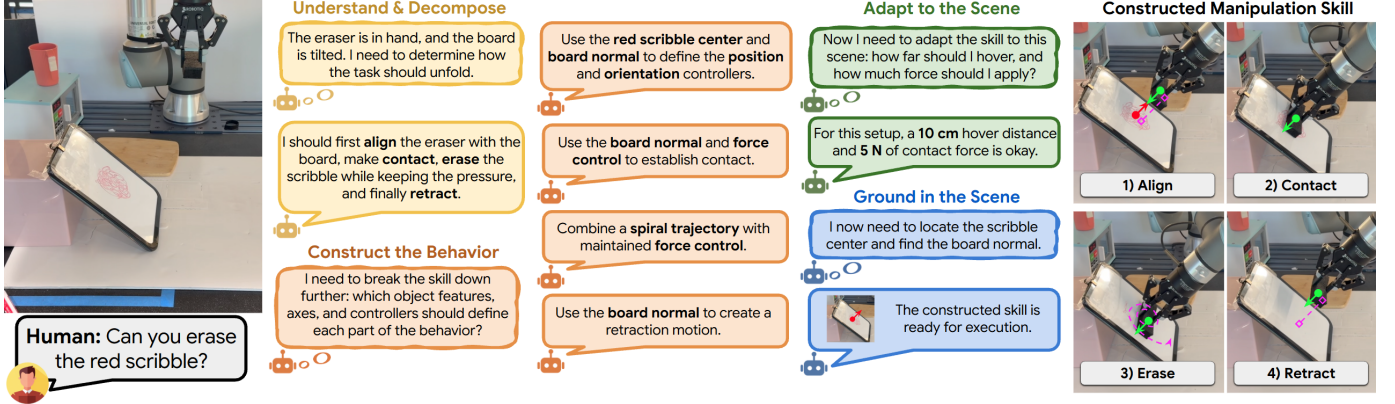


Fig. 1. **Constructing a manipulation skill with GTA-2.** Given the instruction “erase the red scribble,” GTA-2 decomposes the task into alignment, contact, wiping, and retraction, then constructs each part of the behavior by combining task-relevant object features, task axes, and reusable controllers. Scene-specific parameterization and visual groundings instantiate this structure for the observed setup, producing an executable manipulation skill.

Abstract—Robotic manipulation tasks are often decomposed into reusable behaviors or skills, yet these units can remain too coarse to expose the geometric, control, and scene-dependent decisions required for execution. We introduce *Grounded Task Axes version 2* (GTA-2), a modular multi-agent vision-language model (VLM) framework that constructs executable manipulation skills from reusable object-centric task-axis components. Rather than predicting actions end-to-end or relying on predefined task-level skill primitives, GTA-2 explicitly represents manipulation structure through semantic subtasks, task-relevant keypoints and axes, controller compositions, and scene-dependent parameters. Four specialized VLM agents progressively decompose the task, generate an abstract task-axis skill, assign controller parameters, and ground the required visual features in an RGB-D observation. This separation between abstract skill construction and scene-specific instantiation enables zero-shot generation of new manipulation behaviors without task-specific robot demonstrations, policy training, or fine-tuning. Because the intermediate decisions remain explicit, users can also provide targeted feedback to the responsible agent while preserving components of the generated skill that are already correct. We evaluate GTA-2 on a real robot across 14 diverse manipulation tasks and compare it against the generalist VLA policy $\pi_{0.5}$ and two Code-as-Policies baselines using either task-axis controllers or conventional robot primitives. GTA-2 achieves stronger zero-shot execution across the benchmark, while targeted refinement further improves and stabilizes skill generalization across scene configurations.

Index Terms—zero-shot robot manipulation, task-axis controllers, language-conditioned robotics

I. INTRODUCTION

Robotic manipulation tasks are commonly represented as compositions of simpler behaviors or skills, allowing complex tasks to be decomposed into more manageable units such as reaching, grasping, moving, inserting, or placing. However, these skill-level units often remain relatively coarse: each behavior must still encode how the robot should interact with

the objects in the current scene, including which object parts are relevant, how the end-effector should be oriented, and what geometric or contact constraints should be satisfied. We argue that effective open-world manipulation requires decomposing behaviors one level further, into reusable object-centric geometric and controller-level components that can be recombined as needed. For example, as illustrated in Fig. 1, a task such as “erase the red scribble” can be decomposed into alignment, contact, wiping, and retraction. Yet executing these subtasks still requires selecting task-relevant object features such as the scribble center and board normal, composing position, orientation, force, and trajectory controllers, and adapting their parameters to the current scene. Natural language provides an intuitive way to specify such task-level intent, but bridging that intent to execution requires an intermediate representation that explicitly captures this underlying skill structure. Such a representation should be abstract enough to transfer across object instances and configurations, yet explicit enough to be instantiated and executed by low-level robot controllers.

Recent language-conditioned manipulation systems offer two prominent routes toward bridging the gap between task intent and robot execution. Vision-language-action (VLA) models learn generalist behaviors that map visual observations and natural-language commands directly to robot actions [1]–[5]. While these systems have demonstrated broad task capabilities, the internal structure of the learned behavior remains largely implicit, making it difficult to identify how the system decomposes a task or reasons about the scene and execution. Language-model-based programming approaches such as Code-as-Policies instead generate executable robot programs by composing available APIs and predefined skill primitives [6]. However, these predefined primitives typically encapsulate substantial perception, grounding, and control logic, requiring the available skill library to already contain

behaviors capable of solving the task. Thus, while VLAs learn general behaviors and language-programming methods compose predefined behaviors, both largely operate on behaviors as atomic units whose internal structure is fixed or opaque. In contrast, we aim to operate at a **sub-atomic level**, constructing new behaviors on the fly from reusable object-centric geometric and controller-level components.

Task-axis controllers provide a natural representation for operating at this sub-atomic level by expressing manipulation behaviors as structured compositions of reusable controllers grounded in object-centric coordinate frames [7]–[9]. These controllers express motion, orientation, force, and constraint objectives relative to semantically meaningful keypoints and axes, such as a handle axis, a surface normal, an insertion direction, or a grasp point [10]–[15]. Grounded Task-Axes (GTA) [16] builds on this idea by representing manipulation skills through task-relevant keypoints, axes, controller compositions, priorities, and action spaces. This representation makes the structure of the skill explicit: an abstract skill can specify what must be achieved in terms of object-relative geometry, and the same skill can then be instantiated in a new scene by grounding the required keypoints and axes. However, prior task-axis-based systems still required hand-engineered task and feature specifications, including task decomposition, controller selection, task-axis definition, and parameter tuning.

In this work, we introduce *Grounded Task Axes version 2* (GTA-2), a modular multi-agent VLM framework that synthesizes executable, object-centric manipulation skills from a high-level task prompt and a visual observation of the scene. GTA-2 uses the semantic and visual reasoning capabilities of vision-language models to construct task structure from reusable task-axis components, ground object-relative geometry in the observed scene, and adapt controller parameters to scene-specific properties such as object shape, size, pose, material, and state. Rather than generating actions end-to-end, GTA-2 generates an explicit task-axis skill representation that connects language-level task reasoning to controller-level robot execution.

The framework is organized as a multi-agent VLM pipeline that separates abstract skill synthesis from scene-specific grounding. The first two agents construct the abstract skill representation: a **Task Decomposer** converts the user instruction into an ordered sequence of semantic subtasks, and a **Skill Generator** maps each subtask to an abstract skill recipe by selecting the required keypoints, task axes, and controller compositions. The next two agents instantiate this abstract representation in the observed scene: a **Parameter Setter** assigns task- and scene-specific controller parameters such as offsets, forces, and orientation targets, while a **Vision Module** grounds the requested keypoints and axes in the RGB-D observation. The resulting grounded skill specification is then compiled into an executable robot script using a reusable library of task-axis controllers.

Beyond reusability, this structured formulation makes the robot’s intermediate decisions explicit, modular, and refinable. Because GTA-2 exposes the subtask sequence, skill recipe, controller parameters, and visual groundings as separate intermediate outputs, a human can provide targeted feedback

to any stage of the pipeline. For example, if the task should be decomposed differently, the user can refine the Task Decomposer output; if the robot should use a different contact point or motion constraint, the user can refine the Skill Generator or Vision Module; and if the robot should lift an object higher before moving it, the correction can be routed to the Parameter Setter. This agent-level refinement allows the system to preserve correct parts of the generated skill while updating only the components that require modification, providing a practical interface for human-guided adaptation without retraining an end-to-end policy.

We evaluate GTA-2 on a real robot across 14 diverse manipulation tasks and compare it against three baselines: the generalist VLA policy $\pi_{0.5}$, a Code-as-Policies baseline with access to the same task-axis controllers as GTA-2, and a Code-as-Policies baseline using conventional robot motion and force primitives. Our experiments evaluate zero-shot task execution, generalization of generated skills across scene configurations, and targeted human refinement, while the baseline comparisons isolate the benefits of structured skill generation and task-axis controller composition.

II. RELATED WORK

Task-Axis Controllers: Task-axis controllers build on a long line of work that represents manipulation in task-relative coordinate systems rather than as generic joint-space motions or single global world frames. Early task-frame and compliance-control formulations showed that manipulation objectives can be specified relative to task-relevant frames and that different task directions can be regulated through position, force, or compliance objectives [7]–[9]. Subsequent manipulation-planning work used related ideas to define constrained task-space regions, object-centric action spaces, and object-relative planning frames for pose-constrained manipulation, rearrangement, and dynamic environments [11]–[13]. More recent work has explored learning and composing modular object-centric controllers and sequential force-interaction skills, demonstrating that structured controller representations can improve generalization across objects, subtasks, and interaction phases [10], [14]. Task-axis controllers make this structure explicit by parameterizing motion, orientation, force, and constraint objectives with object keypoints and axes; [15] showed that keypoint-grounded task-axis controllers can generalize across variations in object shape, size, and geometry. Grounded Task-Axes (GTA) further extended this formulation to zero-shot semantic skill transfer by using visual foundation models to identify corresponding grounding features on novel objects [16]. Building on this line of work, GTA-2 addresses the remaining dependence on human-designed task decompositions, controller recipes, task-axis definitions, parameter choices, or example skills by using a multi-agent VLM pipeline to generate these directly from a task prompt and scene observation.

LLM-based code generation: LLMs have also been used for robot planning and program generation by translating natural-language goals into sequences of high-level plans or executable code. Systems such as SayCan use LLMs for semantic task reasoning while grounding decisions through

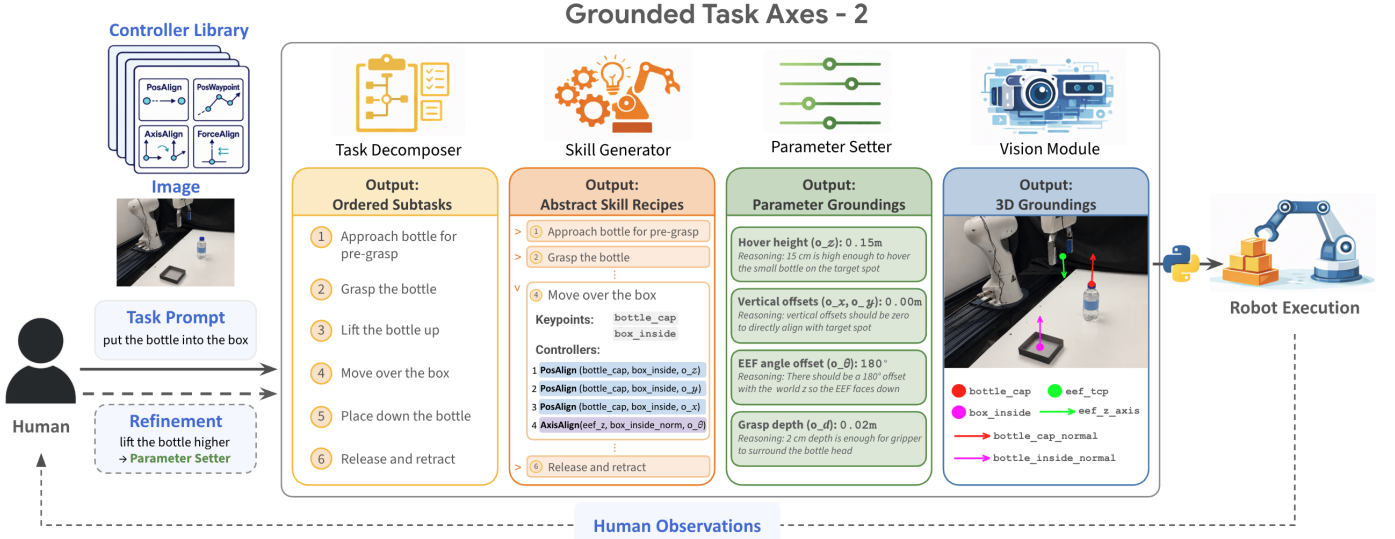


Fig. 2. **Overview of GTA-2.** GTA-2 takes a task prompt, scene image, and controller library as inputs. Specialized agents process these inputs to decompose the task, generate abstract skill recipes, set task-specific parameters, and produce 3D visual groundings. These outputs are combined into an executable robot script. Human feedback provides a refinement loop for updating the subtasks, parameters, or groundings when needed.

affordance models or scene feedback [17], [18]. Related approaches, including ProgPrompt and Code-as-Policies, constrain LLM outputs through program-like prompts, function libraries, perception APIs, or robot-control primitives to produce executable plans or policy code [6], [19]–[23]. These works show that LLMs can provide useful semantic reasoning for robot behavior generation, but they generally depend on a single-step generated program using predefined primitives or perception APIs to bridge language and execution. Our work builds on the same generative insight while shifting the focus from directly coding over existing primitives to generating an explicit task-axis skill representation that can be grounded in the scene and refined through feedback.

Vision Language Action models: Vision-language-action (VLA) models have recently become a major direction for language-conditioned robotic manipulation, mapping visual observations and natural-language instructions directly to robot actions. Systems such as Perceiver-Actor, RT-1, RT-2, Octo, OpenVLA, π_0 , and $\pi_{0.5}$ show that scaling robot policy learning with transformer architectures, large robot datasets, vision-language pretraining, and cross-embodiment data can produce increasingly capable generalist robot policies [1]–[5], [24], [25]. Some recent models, such as $\pi_{0.5}$, also begin to introduce higher-level reasoning by predicting intermediate subgoals before low-level action execution. Nevertheless, the core formulation remains policy-centric: the learned model is optimized to produce actions, and the reasoning that leads to those actions is largely internal to the policy. This can make the behavior difficult to diagnose or adapt when the scene changes, and recent robustness studies show that VLA policies remain sensitive to variations in object layout, viewpoint, robot initial state, lighting, background, physical conditions, and language prompts [26]–[28]. GTA-2 takes a complementary approach: instead of training an end-to-end action policy, it uses VLM agents to construct a structured zero-shot manipulation skill that can be grounded in the current scene, refined through feedback, and compiled into robot execution.

III. METHOD

GTA-2 is a modular multi-agent VLM framework for generating executable manipulation skills from a natural-language instruction ℓ and an RGB-D observation I . Rather than mapping the instruction and observation directly to robot actions, GTA-2 constructs an explicit task-axis skill representation that serves as an intermediate layer between language-level task reasoning and controller-level execution. This representation is generated without task-specific robot demonstrations, policy training, or fine-tuning.

As shown in Fig. 2, the framework separates abstract skill generation from scene-specific grounding. The Task Decomposer first converts the user instruction into an ordered sequence of semantic subtasks. The Skill Generator then maps these subtasks into a lifted task-axis skill $\tilde{S}$ by selecting symbolic object features, task axes, controller compositions, and priorities from a reusable controller library $\mathcal{C}$. The Parameter Setter assigns numerical controller parameters, while the Vision Grounder grounds the required scene-object keypoints and axes from the RGB-D observation. Together, these stages instantiate the lifted skill into a grounded task-axis skill S .

The grounded skill is compiled into an executable robot script that evaluates task-axis controllers at runtime and emits low-level position, orientation, force, and gripper commands through a robot-agnostic execution interface. Because each stage produces an explicit intermediate output, GTA-2 also supports targeted human refinement: a user can correct the task decomposition, skill recipe, parameter values, or visual grounding without regenerating the entire behavior. The following sections define the task-axis skill representation, describe the multi-agent generation and grounding process, and explain the execution and refinement mechanisms.

A. Task-Axis Skill Representation

GTA-2 builds on the task-axis skill representation introduced in Grounded Task-Axes [16], where manipulation behaviors are represented as compositions of reusable controller

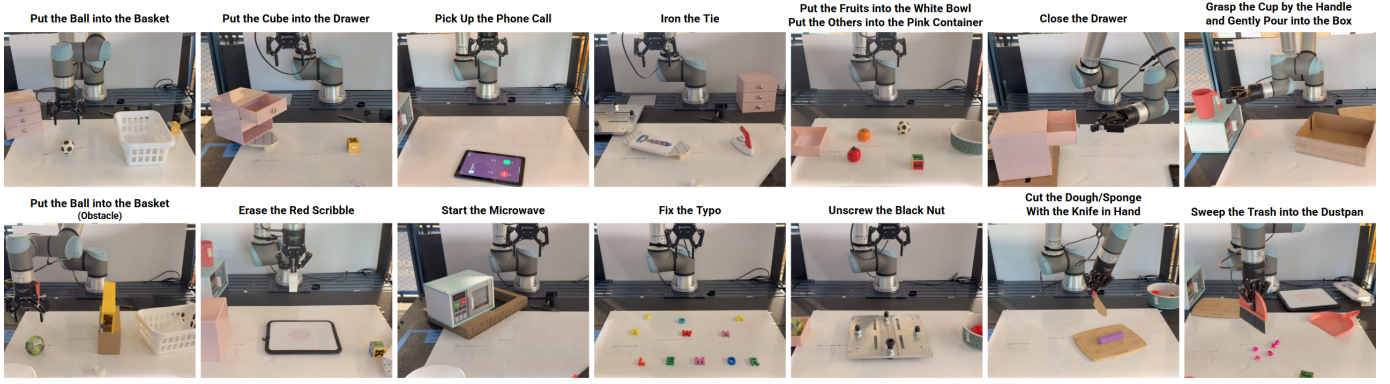


Fig. 3. Representative real-robot executions across the 14 manipulation tasks used in our evaluation. Each example shows key aspects of the task diversity handled by GTA-2, including pick-and-place, articulated-object interaction, tool use, contact-rich manipulation, and constrained motion.

objectives grounded in object-relevant keypoints and axes. In GTA-2, this representation serves as the central intermediate structure between language-level task reasoning and robot-level execution. Given a natural-language instruction ℓ and an RGB-D observation $\mathbf{I}$, the VLM agents do not directly output low-level actions; instead, they generate and ground a structured task-axis skill that can be compiled into executable robot commands.

We represent the scene as a set of objects $\mathcal{O} = \{O_i\}_{i=1}^N$. Each object O_i is associated with a set of semantic keypoints $\mathcal{K}_i = \{k_{i,j}\}$ and axes $\mathcal{A}_i = \{a_{i,m}\}$. Keypoints denote task-relevant 3D locations, such as a drawer handle center, a bottle grasp point, a container rim, or a placement region. Axes denote task-relevant directions, such as a surface normal, a handle direction, an insertion axis, a pouring direction, or an axis between two keypoints. The robot end-effector and the world frame are also represented as objects with their own keypoints and axes, allowing the same representation to express object-relative, robot-relative, and world-relative constraints.

A keypoint or axis can be directly grounded from the RGB-D observation $\mathbf{I}$, derived from other grounded features, or obtained from robot state and calibrated reference frames. We denote a symbolic feature by $\phi \in \mathcal{K}_i \cup \mathcal{A}_i$, which refers to either a keypoint or an axis associated with an object. For scene objects, grounding maps semantic keypoints and axes to metric quantities using perception and point-cloud operations. For the end-effector object O_{eef} , keypoints and axes are obtained from the robot state, forward kinematics, and the calibrated tool frame. For the world object O_{world} , keypoints and axes are defined by the fixed world or robot base frame. We write this grounding map as

$$G_{\mathbf{I},\mathbf{x}}(k_{i,j}) = \mathbf{p}_{i,j} \in \mathbb{R}^3, \quad G_{\mathbf{I},\mathbf{x}}(a_{i,m}) = \mathbf{v}_{i,m} \in \mathbb{S}^2,$$

where $\mathbf{x}$ denotes the robot and system state required for grounding robot-relative and world-relative features.

Derived axes are represented by deterministic feature functions. For example, a surface-normal axis can be computed from a point-cloud neighborhood around a grounded keypoint, an object-relative direction can be computed as an axis between two keypoints, and a third orthogonal direction can be computed as the cross product of two existing axes. This allows the representation to describe geometric constraints

using a small set of semantic features while still producing continuous quantities required for robot control.

A task-axis controller is a parameterized objective defined over these grounded features. We write a lifted controller as

$$\tilde{c} = (\tau, \phi_{\text{ctl}}, \phi_{\text{ref}}, a_{\text{axis}}, \tilde{\theta}, \rho),$$

where τ is the controller type, ϕ_{ctl} is the controlled robot or object feature, ϕ_{ref} is the target feature, a_{axis} is an optional constraint or task axis that the controller operates on, $\tilde{\theta}$ denotes symbolic or unassigned controller parameters, and ρ denotes the priority or execution role of the controller. The controller type τ is selected from a reusable library, including position alignment, axis/orientation alignment, waypoint motion, force objectives, gripper commands, and learned motion primitives when available. Some controllers use all fields in the tuple, while others use only a subset; for example, a gripper command may not require a target keypoint or constraint axis.

Once the symbolic features and parameters are grounded, each controller becomes an executable objective. A grounded controller is written as

$$c = (\tau, G_{\mathbf{I},\mathbf{x}}(\phi_{\text{ctl}}), G_{\mathbf{I},\mathbf{x}}(\phi_{\text{ref}}), G_{\mathbf{I},\mathbf{x}}(a_{\text{axis}}), \theta, \rho),$$

where θ contains numerical parameters such as offsets, distances, angles, forces, heights, tolerances, or gripper states. Each controller type defines an error function

$$\mathbf{e}_c = E_{\tau}(G_{\mathbf{I},\mathbf{x}}(\phi_{\text{ctl}}), G_{\mathbf{I},\mathbf{x}}(\phi_{\text{ref}}), G_{\mathbf{I},\mathbf{x}}(a_{\text{axis}}), \theta),$$

(e axis here) which is evaluated at runtime and converted into execution commands. For example, a position-alignment controller can reduce the displacement between the end-effector TCP and a drawer handle center along a drawer-face normal with a specified offset. An axis-alignment controller can align the gripper approach direction with the drawer face normal while another axis controller constrains the gripper orientation relative to the world vertical axis.

A subtask is represented as a parallel composition of task-axis controllers:

$$\tilde{s}_t = \{\tilde{c}_{t,1}, \dots, \tilde{c}_{t,M_t}\},$$

where the controllers jointly specify the objectives that should be satisfied during that subtask. A single subtask can therefore combine multiple simultaneous constraints, such as aligning the end-effector position with a target keypoint, aligning the

gripper approach axis with an object surface normal, maintaining a desired wrist orientation, and commanding the gripper. A full manipulation skill is an ordered sequence of such subtasks,

$$\tilde{\mathcal{S}} = [\tilde{s}_1, \dots, \tilde{s}_T].$$

This sequence defines the temporal structure of the manipulation behavior, while each subtask defines the simultaneous geometric, force, orientation, or gripper objectives required to accomplish the associated subtask.

We distinguish between lifted and grounded task-axis skills. A lifted skill $\tilde{\mathcal{S}}$ contains symbolic object names, keypoint names, axis names, controller types, and parameter placeholders. It specifies the abstract manipulation structure without committing to a particular scene geometry. A grounded skill $\mathcal{S}$ instantiates this representation by binding each symbolic keypoint to a 3D position, each symbolic axis to a 3D unit vector, and each parameter placeholder to a numerical value. The grounding and parameterization process can be written as

$$\mathcal{S} = (G_{\mathbf{I},\mathbf{x}}, \Theta_{\ell,\mathbf{I}})(\tilde{\mathcal{S}}),$$

where $G_{\mathbf{I},\mathbf{x}}$ grounds symbolic keypoints and axes using perception, robot state, and calibrated frames, and $\Theta_{\ell,\mathbf{I}}$ assigns numerical controller parameters using the task instruction and scene context. In GTA-2, the Vision Module estimates the scene-object components of $G_{\mathbf{I},\mathbf{x}}$, while robot-relative and world-relative features are provided by the robot state and fixed reference frames. The Parameter Setter estimates $\Theta_{\ell,\mathbf{I}}$. This separation allows the system to preserve an abstract skill structure while adapting its grounding and parameters to the current scene.

This representation is useful for GTA-2 because it cleanly separates what the skill is from where and how it should be executed in the current scene. For example, an abstract drawer-opening skill may specify subtasks for approaching the handle, closing the gripper, pulling along the drawer face normal, and releasing the handle. The same lifted structure can be reused for a different drawer, while the Vision Module grounds the handle center and face normal and the Parameter Setter assigns the pull distance, grasp offset, and orientation parameters for the observed scene. During execution, each grounded controller is evaluated at runtime to produce controller errors or commands, allowing the generated skill to remain structured while being executed through a robot-specific backend.

B. Multi-Agent VLM Skill Generation

GTA-2 factorizes zero-shot manipulation skill generation into two objectives: generating a lifted task-axis skill and grounding that skill in the current scene. The first two agents construct the abstract skill structure $\tilde{\mathcal{S}}$ from the task instruction and scene context, while the last two agents instantiate this structure into a grounded skill $\mathcal{S}$ by assigning numerical parameters and grounding the required object features. Each agent receives the task instruction ℓ , the RGB-D observation $\mathbf{I}$, and the structured outputs of previous agents, and produces a structured output rather than free-form robot code.

We write the multi-agent generation process as

$$\mathcal{Q} = A_{\text{TD}}(\ell, \mathbf{I}, \mathcal{C}),$$

$$\begin{aligned} \tilde{\mathcal{S}} &= A_{\text{SG}}(\ell, \mathbf{I}, \mathcal{Q}, \mathcal{C}), \\ \Theta_{\ell,\mathbf{I}} &= A_{\text{PS}}(\ell, \mathbf{I}, \tilde{\mathcal{S}}), \quad G_{\mathbf{I},\mathbf{x}}^{\text{scene}} = A_{\text{VG}}(\mathbf{I}, \tilde{\mathcal{S}}), \\ \mathcal{S} &= (G_{\mathbf{I},\mathbf{x}}, \Theta_{\ell,\mathbf{I}})(\tilde{\mathcal{S}}), \end{aligned}$$

where A_{TD} , A_{SG} , A_{PS} , and A_{VG} denote the Task Decomposer, Skill Generator, Parameter Setter, and Vision Grounder, respectively. The subtask sequence $\mathcal{Q}$ provides the temporal skeleton of the task, $\tilde{\mathcal{S}}$ is the lifted task-axis skill, $\Theta_{\ell,\mathbf{I}}$ assigns numerical controller parameters, and $G_{\mathbf{I},\mathbf{x}}^{\text{scene}}$ denotes the scene-object component of the grounding map. The full grounding map $G_{\mathbf{I},\mathbf{x}}$ also includes robot-relative and world-relative features obtained from robot state, forward kinematics, calibration, and fixed reference frames.

Task Decomposer: The Task Decomposer converts the natural-language instruction into an ordered sequence of semantic subtasks,

$$\mathcal{Q} = A_{\text{TD}}(\ell, \mathbf{I}, \mathcal{C}) = [q_1, \dots, q_T].$$

Each subtask q_t describes a meaningful phase of the manipulation behavior, such as approaching an object, grasping it, moving it along a task-relevant direction, applying force, placing it, or releasing it. The goal of this agent is not to select low-level controllers, but to expose the temporal and semantic structure of the task. For example, for the instruction “open the top drawer,” the Task Decomposer may produce subtasks for moving to the drawer handle, closing the gripper, pulling the drawer outward, and releasing the handle. This decomposition provides the temporal scaffold that the Skill Generator later converts into task-axis controller objectives.

Skill Generator: The Skill Generator maps each semantic subtask q_t to a lifted task-axis skill recipe $\tilde{s}_t$. Given the controller library $\mathcal{C}$, it selects the controller types, symbolic source and target features, task axes, controller priorities, and unassigned parameter names required for each subtask:

$$\tilde{s}_t = A_{\text{SG}}(q_t, \ell, \mathbf{I}, \mathcal{C}) = \{\tilde{c}_{t,1}, \dots, \tilde{c}_{t,M_t}\}.$$

The output remains lifted: it refers to semantic object features such as “*topdrawer_handle_center*”, “*surface_normal(box_bottom)*”, or “*eef_TCP*”, but does not yet require their metric 3D values. This allows the agent to describe the manipulation structure before committing to a specific scene geometry. For instance, an approach subtask for opening a drawer may include multiple parallel controllers such as position alignment between the end-effector TCP and the drawer handle center, axis alignment between the gripper approach axis and the drawer face normal, and an additional orientation constraint with respect to the world vertical axis.

Parameter Setter: The Parameter Setter instantiates the symbolic controller parameters in $\tilde{\mathcal{S}}$ by estimating numerical values from the task instruction and scene context. It defines the parameter assignment map $\Theta_{\ell,\mathbf{I}}$, which replaces each symbolic parameter placeholder $\tilde{\theta}$ with a concrete value θ . These parameters include quantities such as offsets, approach distances, pull distances, placement heights, force magnitudes, orientation angles, waypoint heights, gripper commands, and tolerances. Unlike the Skill Generator, which determines which controllers and features are needed, the Parameter

Setter determines how those controllers should be numerically configured for the observed task and scene. For example, in a drawer-opening skill, the Skill Generator may specify that the drawer should be pulled along the face normal, while the Parameter Setter assigns the pull distance and grasp offsets.

Vision Grounder: The Vision Grounder estimates the scene-object component of the grounding map $G_{I,x}$ for the symbolic keypoints and axes requested by the lifted skill. Rather than grounding all possible object features in the scene, it grounds only the features required by $\tilde{S}$. For semantic keypoints, the Vision Grounder identifies their image-space locations in the RGB observation, which are then converted to 3D positions using the depth image, camera calibration, and point-cloud operations. For axes, it invokes deterministic feature functions, such as computing a surface normal around a grounded keypoint, computing an axis between two keypoints, or computing a cross product between existing axes. Robot-relative and world-relative features, such as the end-effector TCP, gripper axes, and world vertical direction, are not estimated from vision; they are provided by robot state, forward kinematics, calibrated tool frames, and fixed reference frames.

Together, these four agents convert the task instruction and scene observation into an executable grounded skill. The Task Decomposer and Skill Generator determine the lifted manipulation structure $\tilde{S}$, while the Parameter Setter and Vision Grounder instantiate $\tilde{S}$ into S through $\Theta_{\ell,I}$ and $G_{I,x}$. This factorization lets GTA-2 to use VLM reasoning for semantic task understanding and visual grounding while preserving a structured task-axis representation that can be inspected, refined, and executed by the downstream controller system.

C. Robot Script Compilation and Execution

After grounding, GTA-2 deterministically compiles the task-axis skill into an executable script: $P = \Pi(S, C)$, where Π denotes the compiler, C the controller library, and P the resulting program. During execution, for each subtask, the script evaluates the task-axis controllers at each control step and publishes their outputs through separate ROS topics for position, orientation, force, and gripper control. A robot server receives these outputs and applies them through the configured control backend, separating the generated task script from backend-specific execution code. A detailed flow diagram and implementation details are provided in the Appendix, Sec. A.

D. Human-Guided Agent-Level Refinement

Because GTA-2 exposes the intermediate outputs of each stage, users can provide targeted feedback to the specific agent responsible for an error instead of regenerating the entire behavior. For example, if the generated skill moves too close to an object, the user can provide feedback directly to the Parameter Setter. The parameter assignment can then be updated as $\Theta'_{\ell,I} = A_{PS}(\ell, I, \tilde{S}, h)$, where h is user feedback targeted to the Parameter Setter. The revised grounded skill is then obtained by reusing the same lifted skill and visual grounding, $S' = (G_{I,x}, \Theta'_{\ell,I})(\tilde{S})$.

The same principle applies to the other stages: users can refine the Task Decomposer when the subtask sequence is

wrong, the Skill Generator when the controller recipe or selected object features are wrong, and the Vision Grounder when a keypoint or axis is grounded incorrectly. This agent-level refinement provides a practical interface for human-guided adaptation without policy retraining or full skill regeneration. It allows GTA-2 to keep the modular structure of successful parts of the generated skill while updating only the component that caused the failure.

IV. EXPERIMENTAL RESULTS

We instantiate C with the following task-axis controllers.

- **PosAlign** (k_1, k_2, a, o) aligns k_1 with k_2 along a with offset o , while **AxisAlign** (a_1, a_2, a_r, ϕ) aligns a_1 with a_2 by rotating about a_r with angular offset ϕ .
- **PosWaypoint** $(k_1, k_2, a, [o_n])$ follows a sequence of positional offsets, while **AxisWaypoint** $(a_1, a_2, a_r, [\phi_n])$ follows a sequence of angular offsets.
- **ForceConstant** (a, f) maintains force value f along a .
- **GripperControl** (s, t) sets the gripper to state $s \in \{\text{"open"}, \text{"close"}\}$ at normalized subtask time t .
- **PosDMPSpiral** (k, a, r) reproduces a learned spiral of radius r in the plane orthogonal to a .
- **PosDMPLetter** (k, a, letter) reproduces a specified letter trajectory in the same plane and is included as a distractor for task-relevant controller selection.

All controllers can be composed concurrently within a subtask. The same controller library is available across all evaluated tasks.

In our experiments, we used the Gemini 3.1 Pro [29] model for all four VLM agents, so they share the same underlying model and differ only in role-specific instructions, contextual inputs, and structured output representations; we use no task-specific demonstrations or model fine-tuning.

Physical experiments are conducted with a UR5e manipulator equipped with a Robotiq 2F-85 parallel-jaw gripper and a fixed ZED 2i stereo camera calibrated to the robot base frame. RGB observations provide visual context for the VLM agents, while the corresponding point cloud supplies metric 3D keypoints and geometrically derived task axes, including surface normals and directions between grounded keypoints.

A. Baseline Methods

We compare GTA-2 against an end-to-end vision-language-action policy and two few-shot Code-as-Policies (CaP) baselines. All methods receive the same task instructions and scene observations.

VLA baseline ($\pi_{0.5}$): We use $\pi_{0.5}$ as a representative generalist VLA policy for open-world manipulation [5]. Like GTA-2, it uses language and vision and predicts high-level subtasks; but, it ultimately follows a policy-centric formulation that maps observations and instructions to robot actions.

Task-axis controller baseline (CaP-TAC): CaP-TAC is a Code-as-Policies-style baseline in which a VLM directly generates executable robot policy code from the task instruction and scene observation [6]. To isolate the contribution of GTA-2's structured abstraction-to-grounding pipeline, CaP-TAC is given access to the same sub-atomic task-axis controllers. The generated program may compose multiple controllers

GTA-2 Zero-Shot Plan Generation Performance and Feedback Progression



Fig. 4. Zero-shot plan-generation performance and feedback-based recovery of GTA-2 across 14 manipulation tasks. Each task is evaluated over 15 independent runs. Green segments indicate successful executions, while the remaining segments denote runs requiring feedback at that stage and are colored by the module receiving feedback.

concurrently into multiple subtasks. However, CaP-TAC must select perceptual queries, determine the required task geometry, and compose the controller sequence directly within a single generated robot script, without GTA-2’s intermediate task abstractions or structured grounding procedure.

Primitive-action baseline (CaP-Primitive): CaP-Primitive follows the same code-generation and perception protocol as CaP-TAC but replaces task-axis controllers with three conventional robot primitives: absolute Cartesian TCP motion, gripper actuation, and Cartesian motion under a specified contact force. This baseline tests whether direct policy-code generation is sufficient when provided with standard robot primitives, while comparison with CaP-TAC isolates the effect of exposing task-axis controllers.

Both CaP baselines use the same policy-generation model and the vision module (Gemini-3.1-Pro) as GTA-2. Both baselines are supported by a few-shot setting using three fixed demonstrations covering pick-and-place, force-controlled surface contact, and point traversal. The demonstrations are matched in task content across the two APIs and do not include the benchmark tasks themselves

B. Zero-Shot Generation and Targeted Refinement

We first evaluate two central properties of GTA-2: its ability to generate executable manipulation skills without task-specific examples and its ability to refine unsuccessful generations through localized feedback. For each of the 14 tasks, we perform 15 skill-generation runs and validate the resulting skills through physical execution. Each run begins without task-specific feedback. When an execution reveals an error, feedback is directed only to the responsible module, while intermediate representations that remain valid are preserved. The experiment therefore evaluates structured refinement rather than repeated unconstrained regeneration of the complete skill.

As shown in Fig. 4, most zero-shot generations already produce successful executions across the full benchmark, including tasks involving sustained contact and tool use. Although pouring and sweeping require more refinement

than direct transport behaviors, their successful zero-shot runs demonstrate that the generated task-axis representation can express these behaviors without task demonstrations or manually specified skill structures. More importantly, most unsuccessful runs are recovered within two targeted feedback rounds, and all remaining cases are resolved within three. Task complexity therefore increases the amount of refinement required, but does not require new primitives, training, or reconstruction of the complete skill. A small number of high-level corrections is sufficient to turn independently generated failures into successful executions.

The feedback distribution further shows that GTA-2’s agent boundaries correspond to meaningful and independently correctable failure modes. Initial feedback is directed primarily to the Skill Generator, indicating that the task is generally interpreted correctly and that the main remaining challenge is translating this intent into an appropriate task-axis controller composition. Once these structural decisions are corrected, the remaining feedback shifts predominantly to the Vision Module, while no further abstract planning corrections are required. This ordered transition mirrors GTA-2’s abstraction-to-grounding pipeline: it establishes manipulation structure first, after which residual scene-specific grounding errors can be isolated and corrected. The specialized agents therefore provide not only interpretable intermediate outputs but also explicit refinement interfaces, allowing each correction to address the current error source without discarding decisions already resolved by other stages.

C. Task Generalization and Baseline Comparison

We evaluate task generalization success across 14 manipulation tasks, with 20 trials per task and method. *GTA-2 Zero-Shot* executes the initially generated skill without task-specific feedback. In *GTA-2 One-Success*, targeted refinement continues until one successful execution, after which the resulting skill is retained for evaluation. Both settings are compared with $\pi_{0.5}$, *CaP-TAC*, and *CaP-Primitive* under the common protocol described above.

Task Generalization Performances

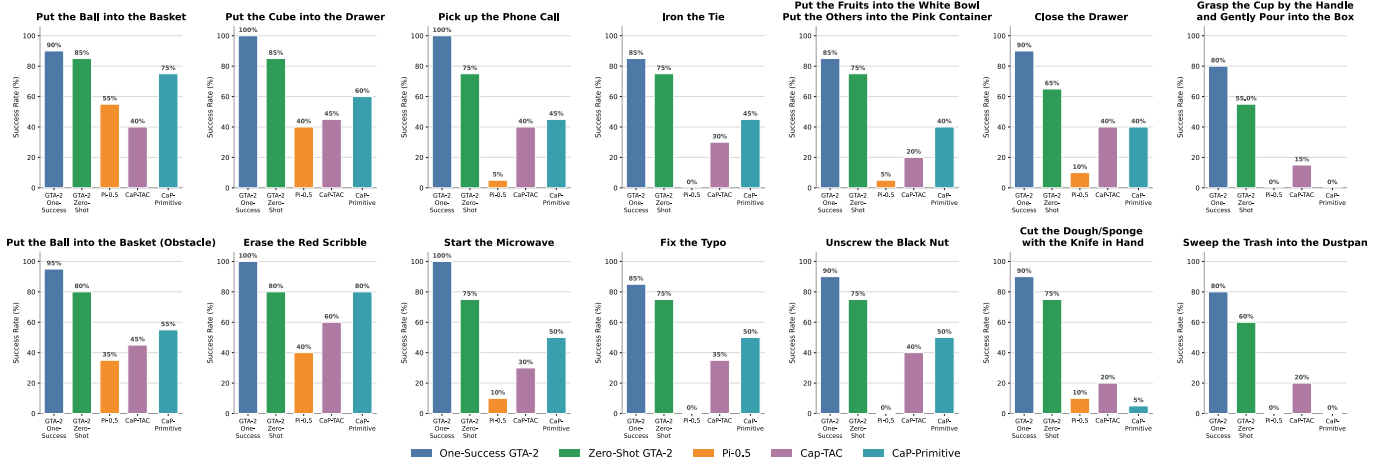


Fig. 5. Task-generalization performance of GTA-2 and the evaluated baselines across 14 manipulation tasks. Bars report success rates for GTA-2 under the One-Success and Zero-Shot settings, together with Pi-0.5, CaP-TAC, and CaP-Primitive. All results are presented on a common 0–100% scale; and evaluated over 20 runs. Values above the bars show the corresponding success rates.

As shown in Fig. 5, GTA-2 Zero-Shot outperforms every baseline while remaining consistent across diverse forms of manipulation. One-Success further improves and stabilizes performance, showing that a correction obtained from a single successful execution produces a skill that remains effective over subsequent trials, rather than correcting only the particular execution in which the feedback was provided.

CaP-Primitive illustrates both the appeal and the limitation of monolithic primitives. By packaging common manipulation behaviors into a small number of function calls, they simplify planning and can perform well when a task matches the expected geometric pattern, particularly for top-down manipulation. However, action sequencing, parameter selection, and visual grounding must still be determined jointly within a single generated program. This coupling makes the resulting policy sensitive to individually small errors within the planning aspect of the program generation. GTA-2 instead assigns these decisions to specialized agents and communicates them through explicit intermediate representations. Its advantage therefore comes not from simplifying the available actions, but from reducing the coupling between reasoning stages, making small errors less likely to invalidate the entire skill with specialized agents.

The comparison between the CaP variants further isolates the generation architecture: CaP-TAC receives GTA-2’s task-axis controller interface but must generate the complete solution within a single program. Since this interface provides a richer and more structured action space than Cartesian primitives, it also demands structured reasoning. Without GTA-2’s specialized agents and intermediate representations, this expressiveness makes generation more difficult, explaining CaP-TAC’s generally lower performance.

Nevertheless, CaP-TAC outperforms CaP-Primitive on pouring, cutting, and sweeping, which depend on contact constraints, object-relative motion, and tool geometry. This suggests that task-axis representations provide a useful inductive bias for contact-rich manipulation and tool use. The results therefore identify two complementary contributions: task-axis

representations provide the manipulation abstraction, while GTA-2’s specialized multi-agent pipeline makes it usable by separately resolving task decomposition, controller composition, parameterization, and visual grounding.

V. CONCLUSION

We presented GTA-2, a modular multi-agent VLM framework that bridges language-level task intent and robot execution through an explicit object-centric task-axis skill representation. By separating task decomposition, controller composition, parameterization, and visual grounding, GTA-2 constructs executable manipulation behaviors from reusable controllers without task-specific demonstrations, policy training, or fine-tuning. This structure also makes the generated behavior inspectable and enables human feedback to be directed to the responsible stage while preserving decisions that are already correct.

Across 14 diverse real-robot manipulation tasks, GTA-2 demonstrated stronger and more consistent zero-shot performance than the evaluated VLA and Code-as-Policies baselines. Targeted refinement efficiently recovered unsuccessful generations and produced skills that remained effective across subsequent scene configurations. The results further show that task-axis controllers are most effective when paired with specialized reasoning stages and explicit intermediate representations. Together, these findings establish GTA-2 as a practical step toward open-world manipulation systems that can construct, inspect, and adapt new behaviors rather than relying on fixed skill primitives or opaque end-to-end policies.

REFERENCES

- [1] M. Shridhar, L. Manuelli, and D. Fox, “Perceiver-actor: A multi-task transformer for robotic manipulation,” in *Conference on Robot Learning*. PMLR, 2023, pp. 785–799.
- [2] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, *et al.*, “Rt-1: Robotics transformer for real-world control at scale,” *arXiv preprint arXiv:2212.06817*, 2022.
- [3] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.

- [4] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, *et al.*, “Octo: An open-source generalist robot policy,” *arXiv preprint arXiv:2405.12213*, 2024.
- [5] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, *et al.*, “ $\pi_{0.5}$: a vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [6] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 9493–9500.
- [7] D. H. Ballard, “Task frames in robot manipulation,” in *AAAI*, 1984.
- [8] M. T. Mason, “Compliance and force control for computer controlled manipulators,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 11, no. 6, pp. 418–432, 1981.
- [9] M. H. Raibert and J. J. Craig, “Hybrid position/force control of manipulators,” 1981.
- [10] S. Manschitz, M. Gienger, J. Kober, and J. Peters, “Learning sequential force interaction skills,” *Robotics*, vol. 9, no. 2, p. 45, 2020.
- [11] T. Migimatsu and J. Bohg, “Object-centric task and motion planning in dynamic environments,” *IEEE RAL*, 2020.
- [12] J. E. King, M. Cagnetti, and S. S. Srinivasa, “Rearrangement planning using object-centric and robot-centric action spaces,” in *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 2016.
- [13] D. Berenson, S. Srinivasa, and J. Kuffner, “Task space regions: A framework for pose-constrained manipulation planning,” *IJRR*, 2011.
- [14] M. Sharma, J. Liang, J. Zhao, A. LaGrassa, and O. Kroemer, “Learning to compose hierarchical object-centric controllers for robotic manipulation,” *arXiv preprint arXiv:2011.04627*, 2020.
- [15] M. Sharma and O. Kroemer, “Generalizing object-centric task-axes controllers using keypoints,” in *IEEE ICRA*, 2021, pp. 7548–7554.
- [16] M. Y. Seker, S. Aggarwal, and O. Kroemer, “Grounded task axes: Zero-shot semantic skill generalization via task-axis controllers and visual foundation models,” in *2025 IEEE-RAS 24th International Conference on Humanoid Robots (Humanoids)*. IEEE, 2025, pp. 791–798.
- [17] B. Ichter, A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog, D. Ho, J. Ibarz, A. Irpan, E. Jang, R. Julian, D. Kalashnikov, S. Levine, Y. Lu, C. Parada, K. Rao, P. Sermanet, A. T. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, M. Yan, N. Brown, M. Ahn, *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” in *Proceedings of The 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 287–318.
- [18] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, T. Jackson, N. Brown, L. Luu, S. Levine, K. Hausman, and B. Ichter, “Inner monologue: Embodied reasoning through planning with language models,” in *Proceedings of The 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 1769–1782.
- [19] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, “Progprompt: Generating situated robot task plans using large language models,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 11 523–11 530.
- [20] S. H. Vemprala, R. Bonatti, A. Bucker, and A. Kapoor, “Chatgpt for robotics: Design principles and model abilities,” *IEEE Access*, vol. 12, pp. 55 682–55 696, 2024.
- [21] Y. Mu, J. Chen, Q.-L. Zhang, S. Chen, Q. Yu, C. Ge, R. Chen, Z. Liang, M. Hu, C. Tao, P. Sun, H. Yu, C. Yang, W. Shao, W. Wang, J. Dai, Y. Qiao, M. Ding, and P. Luo, “Robocodex: Multimodal code generation for robotic behavior synthesis,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 36 434–36 454.
- [22] K. Burns, A. Jain, K. Go, F. Xia, M. Stark, S. Schaal, and K. Hausman, “Genchip: Generating robot policy code for high-precision and contact-rich manipulation tasks,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 9596–9603.
- [23] S. Xie, H. Wang, Z. Xiao, R. Wang, and X. Chen, “Robotic programmer: Video instructed policy code generation for robotic manipulation,” *arXiv preprint arXiv:2501.04268*, 2025.
- [24] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 2679–2713.
- [25] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, “ π_0 : A vision-language-action flow model for general robot control,” in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.
- [26] H. Liu, S. Ruan, J. Long, J. Wu, J. Hou, H. Tang, T. Jiang, W. Zhou, and W. Yao, “Eva-VLA: Evaluating Vision-Language-Action models’ robustness under real-world physical variations,” *arXiv preprint arXiv:2509.18953*, 2025.
- [27] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu, “LIBERO-Plus: In-depth robustness analysis of Vision-Language-Action models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.
- [28] Z. Wang, Z. Zhou, J. Song, Y. Huang, Z. Shu, and L. Ma, “VLATest: Testing and evaluating Vision-Language-Action models for robotic manipulation,” *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 1615–1638, 2025.
- [29] Google DeepMind, “Gemini 3.1 pro: Model card,” <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-1-Pro-Model-Card.pdf>, Feb. 2026, published February 19, 2026.

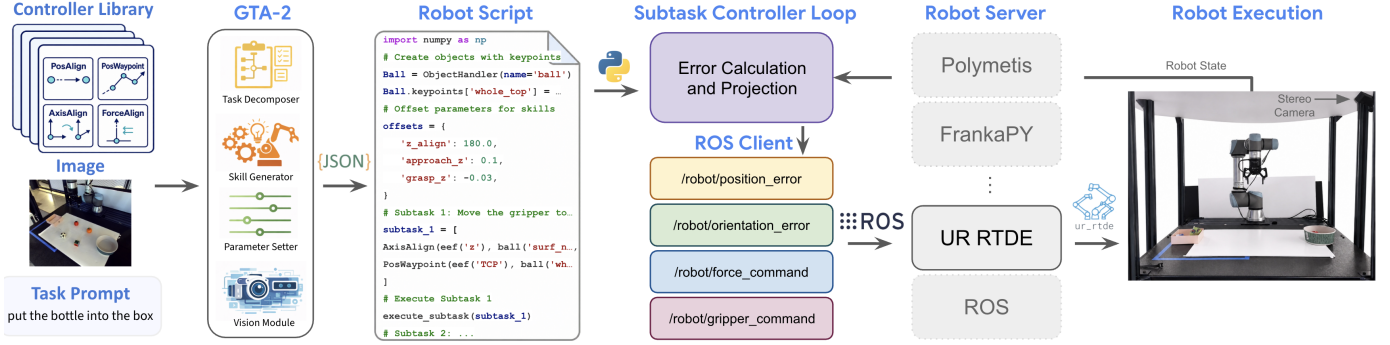


Fig. 6. **Implementation overview of GTA-2.** Given a scene image and a natural-language task prompt, GTA-2 uses a controller library to decompose the task, generate parameterized skills, and produce an executable robot script. Each subtask is executed through a closed-loop controller that computes projected control errors and communicates position, orientation, force, and gripper commands to the robot server via ROS. Robot-state feedback enables closed-loop execution on the physical platform.

TABLE I

QUALITATIVE ERROR MODES OBSERVED IN THE CAP BENCHMARK. THE REPORTED FAILURE MODES ARE BASED ON INSPECTION OF GENERATED PLANS AND ROBOT EXECUTIONS. A TASK MAY EXHIBIT MULTIPLE ERROR MODES.

Task	Observed error modes
Pick Up the Phone Call	Fails to close the gripper at the beginning of the task to use it as a finger; fails to align the end-effector orientation with the phone surface.
Put the Ball in the Basket	Executes the grasp at the hover pose rather than at the object pose.
Close Drawer	Fails to orient the end effector for a lateral pushing motion; the gripper becomes stuck during execution.
Put the Cube into the Drawer	Executes the grasp at the hover pose rather than at the object pose.
Ironing	Fails to plan a force-controlled action; collides the base of the iron with the ironing table while lowering it; fails to align the iron with the surface normal of the ironing table.
Start the Microwave	Selects an unsuitable top-down end-effector orientation when interacting with the microwave.
Ball Pick-and-Place with Wall	Does not use obstacle keypoints to determine a collision-free lifting height; instead estimates the required clearance from visual input, which can result in a collision with the wall.
Fruits and Balls	Places fruits on top of one another, crushing them; collides with the side of the bowl; selects the object center rather than a reachable surface point as the grasp target.
Fix the Typo	Fails to remove the incorrect letter before inserting a replacement; produces invalid target words (e.g., "SYKE" and "YETCH"); collides with the table because of inaccurate placement offsets.
Unscrew the Nut	Confuses clockwise (CW) and counterclockwise (CCW) rotation directions; applies insufficient upward force to remove the nut; rotates the nut by less than one full revolution.
Cutting	Aligns the end effector rather than the knife tip with the dough; does not use knife-specific keypoints; fails to rotate the knife according to the dough orientation; fails to use force control and collides with the table; in other cases, applies excessive force (up to 30 N).
Grasp and Pour	Attempts to grasp the cup from above rather than using a side grasp; generates motions outside the robot workspace.
Erase the Red Scribble	Fails to align the wiping tool with the orientation of the target surface.

APPENDIX

A. CaP Benchmark Qualitative Error Analysis

Table I summarizes recurring error modes observed through inspection of the plans and robot executions produced for the CaP benchmark. The observed errors involve grasp selection, end-effector alignment, contact-force reasoning, object-specific keypoints, collision avoidance, and task-level sequencing.